

DANMARKS TEKNISKE UNIVERSITET



02477 Bayesian machine learning

ASSIGNMENT 3

Gabriel Agerholm Ruge, s214615
Andreas Bagge, s214630

May 17, 2026

Part 1

We are given the following conditional likelihood:

$$p(y_n|x_n, z_n, \mathbf{w}_0, \mathbf{w}_1, \sigma_0^2, \sigma_1^2) = \mathcal{N}(y_n|\mathbf{w}_{z_n}^T \mathbf{x}_n, \sigma_{z_n}^2)$$

Where $z_n|\pi_n \sim \text{Ber}(z_n|\pi_n)$ and $\pi_n = \sigma(\mathbf{v}^T \mathbf{x}_n)$. We also have the following generic prior:

$$\begin{aligned} \tau^2, \sigma_0^2, \sigma_1^2 &\sim \mathcal{N}_+(0, 1) \\ \mathbf{w}_0, \mathbf{w}_1, \mathbf{v} &\sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I}) \\ z_n|\mathbf{v} &\sim \text{Ber}(\sigma(\mathbf{v}^T \mathbf{x}_n)) \\ y_n|z_n &\sim \mathcal{N}(\mathbf{w}_{z_n}^T \mathbf{x}_n, \sigma_{z_n}^2) \end{aligned}$$

The text is inconsistent as to whether it refers to the variances $\tau^2, \sigma_0^2, \sigma_1^2$ or the corresponding standard deviations. It frequently mixes up the two. We have therefore chosen to consistently model the variances with half-normal priors. This improves clarity and if anything only impacts the optimization.

Using this model, we get the joint model on the form:

$$p(\mathbf{y}, \mathbf{w}_1, \mathbf{w}_0, \mathbf{v}, \mathbf{z}, \tau, \sigma_0, \sigma_1) = \dots$$

Task 1.1: Marginalize out each z_n from to joint model

We can marginalize out z_n using the sum rule. Therefore, we write out the following joint distribution:

$$\begin{aligned} p(y_n, z_n|x_n, \mathbf{w}_0, \mathbf{w}_1, \mathbf{v}, \sigma_0, \sigma_1) &= p(y_n|x_n, z_n, \mathbf{w}_0, \mathbf{w}_1, \sigma_0, \sigma_1)p(z_n|\mathbf{v}) \\ &= \mathcal{N}(y_n|\mathbf{w}_{z_n}^T \mathbf{x}_n, \sigma_{z_n}^2)\text{Ber}(z_n|\sigma(\mathbf{v}^T \mathbf{x}_n)) \end{aligned}$$

Now, we can use the sum rule by summing over all possible values of z_n , i.e., $z_n = \{0, 1\}$:

$$\begin{aligned} p(y_n|x_n, \mathbf{w}_0, \mathbf{w}_1, \mathbf{v}, \sigma_0, \sigma_1) &= \sum_{z_n \in \{0, 1\}} p(y_n, z_n|x_n, \mathbf{w}_0, \mathbf{w}_1, \mathbf{v}, \sigma_0, \sigma_1) \\ &= \sum_{z_n \in \{0, 1\}} \mathcal{N}(y_n|\mathbf{w}_{z_n}^T \mathbf{x}_n, \sigma_{z_n}^2)\text{Ber}(z_n|\sigma(\mathbf{v}^T \mathbf{x}_n)) \\ &= (1 - \sigma(\mathbf{v}^T \mathbf{x}))\mathcal{N}(y_n|\mathbf{w}_0^T \mathbf{x}_n, \sigma_0^2) + \sigma(\mathbf{v}^T \mathbf{x})\mathcal{N}(y_n|\mathbf{w}_1^T \mathbf{x}_n, \sigma_1^2) \end{aligned}$$

Finally, we can write joint model without marginalized over $\mathbf{z}$:

$$\begin{aligned} \sum_{\mathbf{z}} p(\mathbf{y}, \mathbf{w}_1, \mathbf{w}_0, \mathbf{v}, \mathbf{z}, \tau, \sigma_0, \sigma_1) &= \sum_{\mathbf{z}} p(\mathbf{y}|\mathbf{w}_1, \mathbf{w}_0, \mathbf{v}, \mathbf{z}, \tau, \sigma_0, \sigma_1)p(\mathbf{v})p(\mathbf{w}_1)p(\mathbf{w}_0)p(\mathbf{z})p(\tau)p(\sigma_0)p(\sigma_1) \\ &= \left(\prod_{n=1}^N (1 - \sigma(\mathbf{v}^T \mathbf{x}_n))\mathcal{N}(y_n|\mathbf{w}_0^T \mathbf{x}_n, \sigma_0^2) + \sigma(\mathbf{v}^T \mathbf{x}_n)\mathcal{N}(y_n|\mathbf{w}_1^T \mathbf{x}_n, \sigma_1^2) \right) \\ &\quad \mathcal{N}(\mathbf{w}_0|\mathbf{0}, \tau^2 \mathbf{I})\mathcal{N}(\mathbf{w}_1|\mathbf{0}, \tau^2 \mathbf{I})\mathcal{N}(\mathbf{v}|\mathbf{0}, \tau^2 \mathbf{I})\mathcal{N}_+(\tau^2|0, 1)\mathcal{N}_+(\sigma_0^2|0, 1)\mathcal{N}_+(\sigma_1^2|0, 1) \end{aligned}$$

Where $\sum_{\mathbf{z}}$ represents the sum over all possible outcomes $\mathbf{z}$.

Task 1.2: Implement a Python function to evaluate the marginalized log joint distribution

We have made two different implementations. See implementations below.

```

1 def log_joint(theta):
2     assert(len(theta) == num_params)
3
4     tau = theta[0] # All these parameters denote the variance. ...
5     # The assignment is inconsistent with the hyperparameters.
6     sigma0 = theta[1]
7     sigma1 = theta[2]
8
9     w0 = theta[3:5]
10    w1 = theta[5:7]
11    v = theta[7:9]
12
13    if jnp.any(theta[:3] < 0):
14        return -jnp.inf
15
16    # log hyperprior
17    log_hyperprior = jnp.sum(log_half_npdf(theta[:3], 0, 1), axis =...
18                             -1)
19
20    # log prior
21    log_prior = jnp.sum(log_npdf(theta[3:], 0, tau))
22
23    # log likelihood p(y|w)
24    sig = sigmoid(X_train@v) # (N,)
25    f0 = X_train@w0
26    f1 = X_train@w1
27
28    models = jnp.hstack(((log_npdf(y, f0, sigma0) + jnp.log(1 - sig...
29                          )[:, None], (log_npdf(y, f1, sigma1) + jnp.log(sig))[:, None]))
30
31    log_likelihood = jax.scipy.special.logsumexp(models, axis=-1)...
32    sum()
33
34    return log_likelihood + log_prior + log_hyperprior

```

```

1 def marginalized_log_joint(x: jnp.ndarray, y: jnp.ndarray, params: ...
  jnp.ndarray) -> float:
2     w00, w01, w10, w11, v0, v1, tau, sigma0, sigma1 = params
3
4     if tau ≤ 0.0 or sigma0 ≤ 0.0 or sigma1 ≤ 0.0:
5         return -jnp.inf
6
7     w0 = jnp.array([w00, w01])
8     w1 = jnp.array([w10, w11])
9     v = jnp.array([v0, v1])
10
11     K = log_npdpf(w0, 0.0, tau**2).sum() + \
12         log_npdpf(w1, 0.0, tau**2).sum() + \
13         log_npdpf(v, 0.0, tau**2).sum() + \
14         log_pos_npdpf(tau**2, 0.0, 1.0) + \
15         log_pos_npdpf(sigma0, 0.0, 1.0) + \
16         log_pos_npdpf(sigma1, 0.0, 1.0)
17
18     logits = x @ v
19     log_p1 = jax.nn.log_sigmoid(logits)
20     log_p0 = jax.nn.log_sigmoid(-logits)
21
22     log_y_z0 = log_npdpf(y, x @ w0, sigma0**2)
23     log_y_z1 = log_npdpf(y, x @ w1, sigma1**2)
24
25     log_mix = logsumexp(jnp.stack([log_p0 + log_y_z0, log_p1 + ...
  log_y_z1]), axis=0)
26     A = log_mix.sum()

```

Task 1.3: Run a Metropolis-Hasting sampler to infer all parameters.

We have used the Metropolis-Hasting algorithm from the exercises. The resulting trace can be seen in Figure 1. In this figure, I have also included the warm-up period. When calculating the posterior mean I remove the first 500 samples from the trace. I have visualized the posterior means in the next exercise, see Figure 3. As a sanity check, I have visualized the fit using the posterior means of the parameters in Figure 2.

Task 1.4: Report the posterior mean and 95% credibility intervals for all parameters.

We have visualized the posterior means and 95% credibility intervals for the parameters in Figure 3. The warm-up period is also excluded in these histograms.

Task 1.5: For $x \in [-12, 12]$, plot the posterior predictive distributions for $p(\pi^*|\mathbf{y}, \mathbf{x}^*)$, $p(y^*|\mathbf{y}, \mathbf{x}^*, z^* = 0)$, and $p(y^*|\mathbf{y}, \mathbf{x}^*, z^* = 1)$

The distributions are visualized in Figure 4. The distributions are visualized using the parameters from the Metropolis-Hastening algorithm.

Task 1.6: For $x \in [-12, 12]$, plot posterior predictive distribution for $p(y^*|\mathbf{y}, \mathbf{x}^*)$

I have visualized the predictive distribution $p(y^*|\mathbf{y}, \mathbf{x}^*)$ on Figure 5.

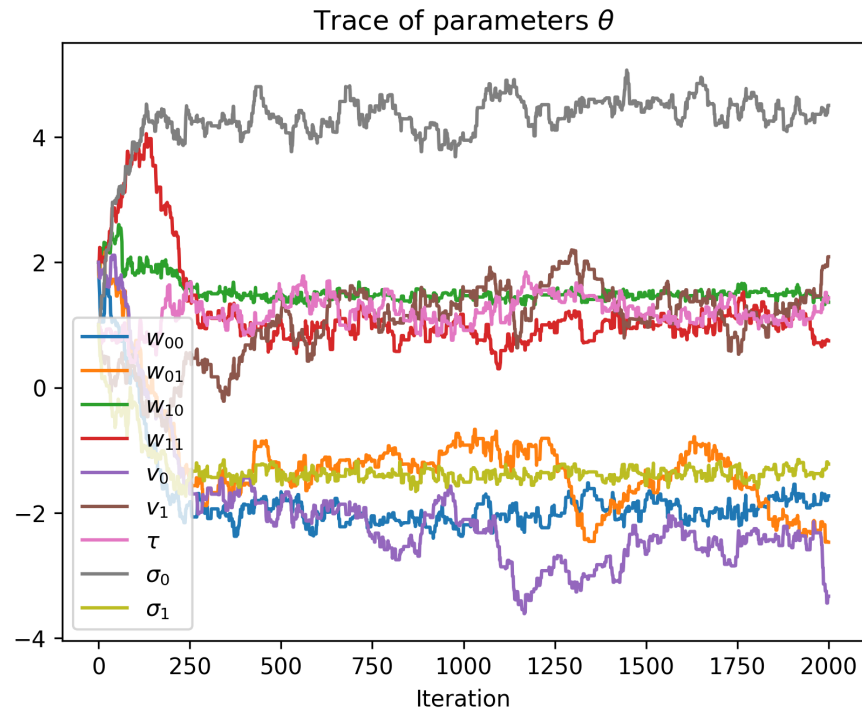


Figure 1: Trace of the parameters of the model using the Metropolis-Hasting algorithm. This plot includes the warm-up.

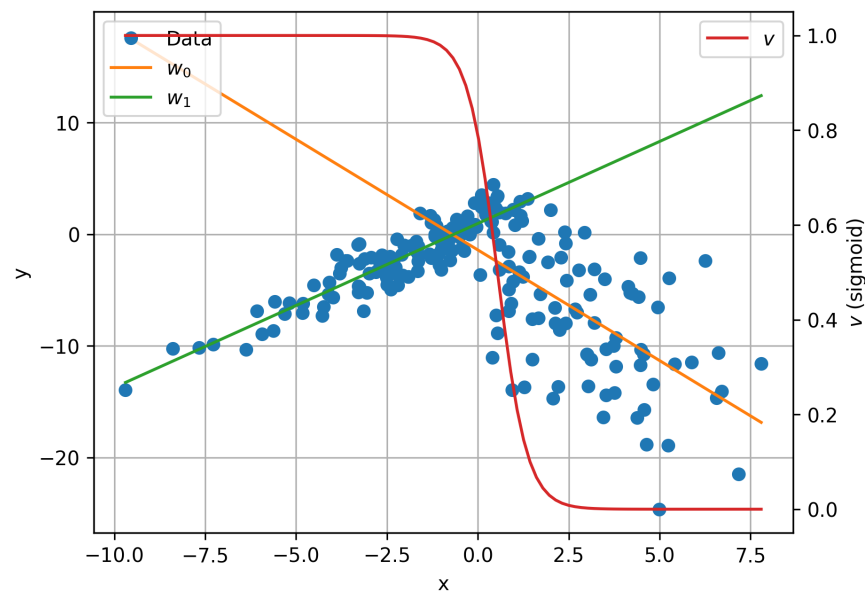


Figure 2: Visualization of the fit using the posterior means of the parameters w_0 , w_1 , and v .

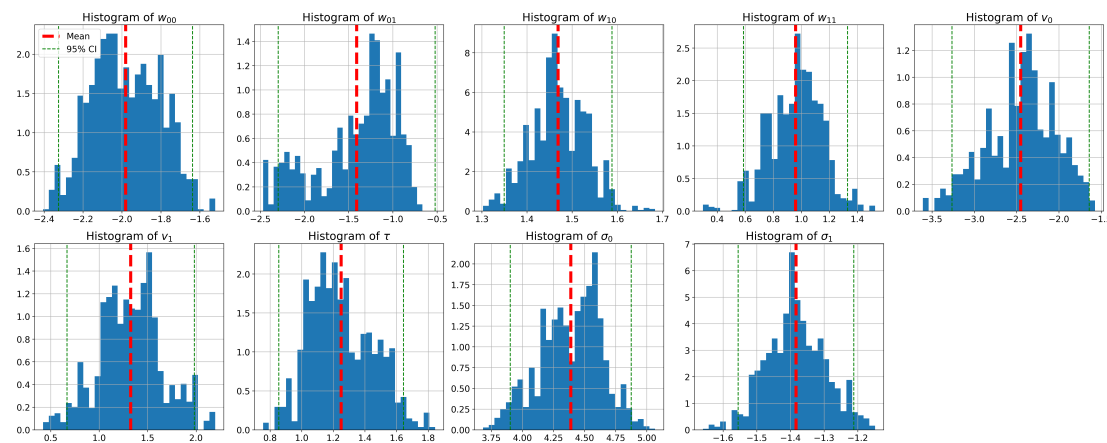


Figure 3: Histograms for the different parameters of the model, including the posterior means and 95% credibility intervals.

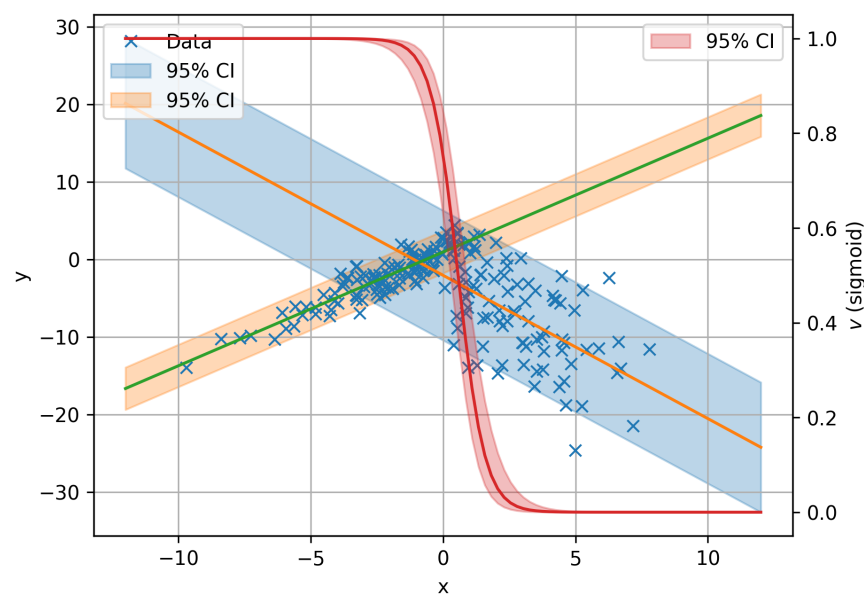


Figure 4: The posterior predictive distributions for $p(\pi^*|\mathbf{y}, \mathbf{x}^*)$, $p(y^*|\mathbf{y}, \mathbf{x}^*, z^* = 0)$, and $p(y^*|\mathbf{y}, \mathbf{x}^*, z^* = 0)$

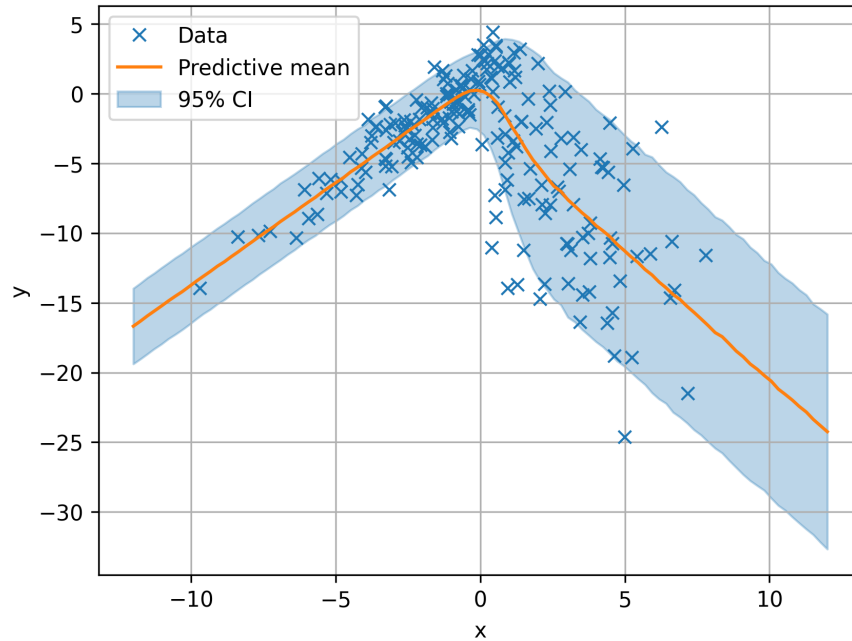
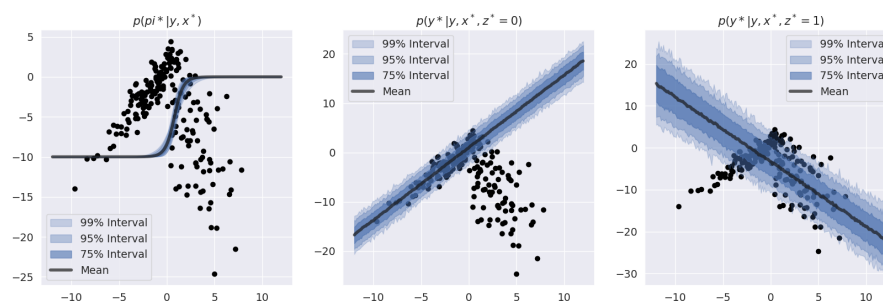


Figure 5: The posterior predictive distribution for $p(y^*|\mathbf{y}, \mathbf{x}^*)$

Below is the same thing performed independently by another student.



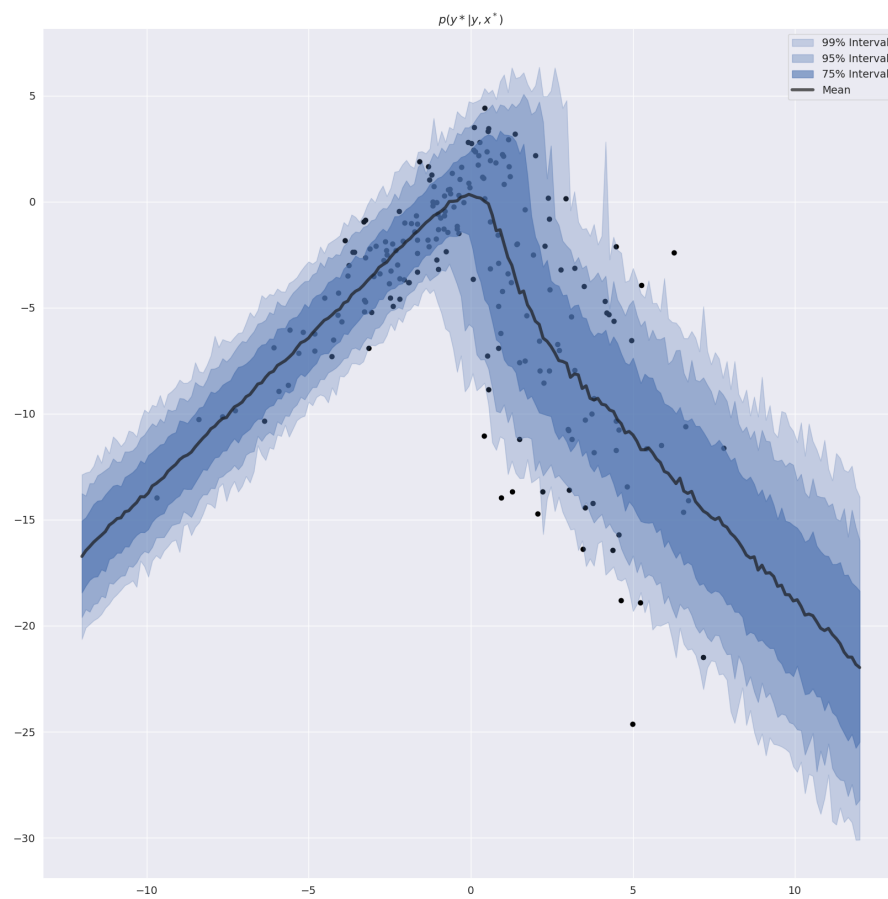


Figure 6: Posterior predictive distribution of y^*